Indian Institute of Technology, Jodhpur

Department Of CSE

PCS II

Lab: CPU Scheduling in xv6

Course Instructor: Dr Sidharth Sharma

Pre requisite-

• Make sure you have already downloaded, installed, and run the **xv6 OS code** provided to you. The OS runs on an x86 emulator called **QEMU**.

In the xv6 directory run:

```
make
make qemu
```

to boot xv6 and open the shell.

• For this lab, you will need to understand the following files in the xv6 source code.

```
proc.c
proc.h
defs.h
sysproc.c
trap.c
```

-The file **proc.c** contains the implementation of process management and the CPU scheduler.

-**proc.h** contains the process structure definition.

-**sysproc.c** contains system call implementations related to process control.

-**trap.c** contains trap and interrupt handling logic.

Before beginning the project, study how the **default scheduler** in xv6 works.

• Learn how to write simple **user programs in xv6**.

Any user program must be added to the **Makefile** so that it can be compiled and executed inside xv6.

Remember that xv6 does not contain a text editor or compiler inside the OS, so you must write programs on the host machine and compile them before running xv6.

Part A: Understanding the default scheduler

In this part you will explore the default scheduling mechanism used by xv6.

The xv6 scheduler repeatedly scans the process table and selects a **RUNNABLE** process to execute.

Tasks:

1. Locate the function **scheduler()** in **proc.c**.
2. Understand how xv6 selects the next process to run.
3. Identify where the **context switch** between processes happens.
4. Observe how the scheduler behaves when multiple processes are running.

Write a small user program that creates several processes using **fork()** and observe how they are scheduled.

Part B: Implementing a priority-based scheduler

In this part you will modify xv6 to support **priority scheduling**.

Each process will have an associated **priority value**, and the scheduler should always select the process with the highest priority among runnable processes.

Tasks:

1. Extend the **process structure** to include a priority field.
2. Modify the scheduler so that it selects the runnable process with the highest priority.
3. Ensure that the scheduler still works correctly when multiple processes are present.
4. If multiple processes have the same priority, the scheduler may select any one of them.

Part C: Adding a system call to change priority

You will now implement a system call that allows a user program to modify the priority of a process.

The system call should have the following format:

```
setpriority(pid, value)
```

where

- **Pid** is the process ID
- **Value** is the priority level

Tasks:

1. Define the new system call.
2. Add the system call to the syscall table.
3. Implement the system call logic in the kernel.
4. Ensure that invalid inputs are handled properly.

Part D: Testing the scheduler

Write test programs that create multiple processes and assign different priorities to them.

Example test scenario:

• Create three processes
• Assign different priorities
• Print messages in a loop from each process

Observe the order in which the processes execute.

Your scheduler should demonstrate that higher priority processes are selected more frequently.

Hints

• Look at how existing system calls such as **getpid()** are implemented.

• The process table can be accessed through the global process array in xv6.

• Ensure that only processes in the **RUNNABLE** state are considered by the scheduler.

- Use **cprintf()** to print debugging information if required.

Testing

Write your own test programs to verify that the scheduler behaves as expected.

You may test the following:

- multiple processes with the same priority
- processes with different priorities
- processes created using **fork()**

Observe the scheduling behavior and verify that it matches your expectations.